



Faculty of Computing

Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BESE 16AB • Semester: Fall 2026

Experiment No. 5

Implementation of General Trees

Date	09 October 2026
Time	09:00 AM–11:50 AM & 2:30 PM–5:00 PM
Instructor(s)	Dr. Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm. CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry.:

1 Introduction

This lab introduces general trees, hierarchical relationships, basic tree operations, and traversal methods using C++ pointers and dynamic memory allocation.

2 Objectives

Implement a general tree; identify parent, children, and leaf nodes; calculate size, height and depth; perform tree traversals; and delete a subtree safely.

3 Tools and Software Requirements

Software: Visual Studio (C++)

Helping Material: Lecture 14: Trees; textbook; previous linked-list and recursion labs.

4 Description

A rooted tree contains nodes connected by parent–child relationships. Unlike a binary search tree, a general tree may have more than two children per node. Use the following node structure:

Listing 1. TreeNode

```
1 struct TreeNode {  
2     int data;  
3     TreeNode* parent;  
4     vector<TreeNode*> children;  
5 };
```

Use `#include <vector>`. Store unique integer values so that a parent or node can be identified unambiguously.

5 Lab Tasks

Task 1: General Tree Construction

Create a root and implement insertion of a new child under a specified parent. Build a tree with at least 10 nodes and display each node with its parent and children.

Task 2: Basic Tree Operations

Implement functions to determine whether a node is the root or a leaf, find its degree, and calculate the total number of nodes in the tree. Test the functions on your tree.

Task 3: Tree Height and Depth

Implement a recursive function for tree height and a function for the depth of a specified node. Use root depth = 0, leaf height = 0, and empty-tree height = −1.

Task 4: Tree Traversals

Implement breadth-first traversal using a queue, and pre-order and post-order depth-first traversals using recursion. Display the visited node values for each traversal.

Task 5: Node Deletion

Implement deletion of a selected node together with all its descendants. Remove its pointer from its parent's children list and free all nodes in the deleted subtree. If the root is selected, delete the entire tree.

and set root to `nullptr`. Display the tree before and after deletion.

Task 6: Tree Analysis

For the original tree and again after deletion, record the number of nodes, height, leaf-node count, and traversal outputs. State the Big-O time complexity of breadth-first, pre-order, and post-order traversals.

6 Important Note

Handle an empty tree and invalid node values. Avoid memory leaks. Deletion in this lab removes the entire selected subtree; it does not reattach its children.

7 Deliverables

Complete the solution section with C++ code, execution screenshots, and Task 6 analysis. Submit one Word document on LMS and demonstrate the working program during the lab.

8 Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)	Marks
Software Problem Re- alization (CLO2)	Unable to identify or analyze the required runtime complexity.	Identifies most complexities with some guidance.	Correctly analyzes operations and justifies runtime complexity.	
Software Tool Usage (CLO3)	Code is incomplete or does not produce required results.	Code is mostly functional with minor issues.	Code is complete, functional, documented and produces correct results.	